



Retrieval and Reasoning Architecture

Technical White Paper

How Verwaltungsassistent retrieves candidate evidence through hybrid search, verifies it independently, and refuses to answer when the evidence is insufficient.

Document type: Technical White Paper

Version: 2.0

Date: 12 September 2026

Status: Public

Deployment reference: Verwaltungsassistent — municipal pilot (Berlin demo corpus)

Audience: municipal/public administration IT architects, enterprise decision makers, partners, investors, technical evaluators

Document Control

Title	Verwaltungsassistent — Technical White Paper: Retrieval and Reasoning Architecture
Version / Date	2.0 / 2026-09-12
Classification	Public
System described	Enterprise Knowledge Reasoning Platform (Verwaltungsassistent), retrieval, evidence-verification and answer-generation architecture, as implemented in the current code base (Java 21, Spring Boot 3.3)
Reference deployment	Verwaltungsassistent — municipal assistant for German public administration (demo corpus: Berlin Service-Portal documents, BRKG, AV § 55 LHO, TV-L 2025)
Basis	Direct inspection of the current implementation (September 2026). All described capabilities are implemented and running; future work is explicitly labelled as such.
Conventions	monospace = implementation identifier; <i>italic</i> = German system wording with translation

ABOUT THIS DOCUMENT

This white paper describes the retrieval and reasoning architecture of the Enterprise Knowledge Reasoning Platform (Verwaltungsassistent) as it is implemented today. It is written for readers without access to the source code, but every architectural statement is grounded in the current implementation. Where a capability is configurable, the default configuration of the reference deployment is described and alternatives are noted. Where a capability is planned but not implemented, it is explicitly labelled *future work*.

THE CENTRAL ARCHITECTURAL CLAIM

Verwaltungsassistent is not a simple vector-RAG system. Its pipeline distinguishes two very different things that naive retrieval systems conflate: **candidate evidence** — material that was *retrieved* because it looked relevant — and **verified evidence** — material that an independent verification stage has confirmed actually *supports* the statements of the generated answer. Every retrieved document is treated as a candidate only. And when verification fails, Verwaltungsassistent fails closed: it says that it does not know, instead of presenting a plausible-sounding answer.

Table of Contents

1	Executive Summary	4
2	Problem: Why Traditional RAG Is Not Enough	5
3	Verwaltungsassistent Architecture Overview	6
4	Document Ingestion	8
5	Hybrid Retrieval: Vector, Lexical and Graph Search	10
6	Intent and Domain Analysis	14
7	Candidate Ranking: Fusion, Reranking and Evidence Selection	15
8	The Critical Evidence Problem	17
9	Evidence Verification	18
10	Fail-Closed Answer Boundary	20
11	Rule-First Processing and Structured Knowledge	21
12	End-to-End Examples	23
13	Performance Considerations	25
14	Auditability and Explainability	26
15	Security, On-Premise and Air-Gapped Deployment	27
16	Comparison with Conventional RAG	28
17	Limitations	29
18	Future Directions	30
19	Conclusion	31
	Glossary	32
	References and Source Notes	34

1 Executive Summary

The Enterprise Knowledge Reasoning Platform (Verwaltungsassistent) is a domain-oriented question-answering and decision-support platform for public administration. It answers natural-language questions over an organization's authoritative document corpus — laws, regulations, service descriptions, procedural guidelines — and, where a question has a deterministic answer in structured knowledge (tariff tables, travel-allowance rules, procurement thresholds), it computes that answer directly by rule instead of guessing with a language model.

The reference deployment described in this document is the **Verwaltungsassistent**, a German municipal assistant currently running a real pilot corpus of official Berlin administration documents. The platform itself is domain-agnostic: all municipal knowledge — domains, terminology, structured rule tables — is supplied as data by the application layer. In the municipal deployment, requests also arrive as citizen and caseworker email: an intake pipeline classifies each incoming message, links it to a case (*Vorgang*) and feeds it through the same retrieval, verification and fail-closed path described in this document.

Verwaltungsassistent's question-answering path consists of six architectural stages:

1. **Intent and domain analysis** — a semantic parser extracts a structured intent (domain, intent type, numeric parameters, language) from the question.
2. **Rule-first routing** — if structured knowledge can answer the question deterministically, it does so (no retrieval, no guessing). Otherwise the question proceeds to retrieval.
3. **Hybrid retrieval** — three complementary mechanisms run in parallel: semantic (vector) search, lexical (keyword) search and graph search. Their results are fused into a candidate pool.
4. **Candidate ranking** — the pool is reranked, adjusted by domain relevance, and compressed into a compact evidence package (at most four documents, three excerpts each).
5. **Independent evidence verification** — a separate verifier model judges, for each claim of the drafted answer, whether the retrieved excerpts actually support it. Retrieval candidates are explicitly *not* treated as evidence by default.
6. **Fail-closed presentation** — the answer is presented only if it is grounded in verified evidence. Otherwise Verwaltungsassistent answers: „Für diese Frage liegen in der Wissensbasis keine ausreichenden Informationen vor.“ ("The knowledge base does not contain sufficient information for this question.")

THE KEY DISTINCTION

RETRIEVED ≠ VERIFIED. Retrieval produces *candidate evidence* — material that looks relevant. Only the verification stage turns candidates into *supporting evidence* for a specific answer. Semantic similarity to the question is a retrieval signal, not proof of support.

THE KEY SAFETY PRINCIPLE

If the evidence is insufficient, Verwaltungsassistent says so. When verification finds no supported claim, the generated answer is not shown. Retrieval failure is never converted into fabricated certainty. This fail-closed boundary is enforced in code, not by prompt instruction alone.

These two principles — the candidate/evidence distinction and the fail-closed boundary — are the architectural core of Verwaltungsassistent and the main subject of this white paper.

2 Problem: Why Traditional RAG Is Not Enough

Retrieval-augmented generation (RAG) — retrieving text passages and handing them to a large language model (LLM) as context — is the standard pattern for grounding model output in an organization's documents. In its conventional form it has four structural weaknesses that matter acutely in public administration, where an answer carries legal and procedural consequences:

2.1 Semantic similarity measures topical relatedness, not evidentiary support

Vector search ranks passages by the cosine distance between the question embedding and passage embeddings. A passage can be *extremely similar* to a question and still not contain the information needed to answer it. A question about an exception to a rule ("registration without a landlord's confirmation") retrieves passages about registration in general — highly similar, entirely unhelpful for the specific exception. Conventional RAG systems feed such passages to the LLM anyway; the LLM then plausibly bridges the gap, and the missing information is filled in by the model's world knowledge. The result reads confidently and cites real documents that do not actually support the answer.

2.2 No independent check between generation and presentation

In conventional RAG, the same retrieved passages that seed generation are also presented as the "sources". There is no step that asks: does this passage actually entail what the answer claims? A citation is treated as proof of support. But *a citation alone does not establish support* — it establishes only that the passage was in the context window.

2.3 No fail-closed behavior

When retrieval returns poor or irrelevant material, a conventional RAG system still generates an answer — the model is instructed to answer, so it answers. A system intended for administrative use must instead be able to say "insufficient information" as a designed, first-class outcome.

2.4 One retrieval mechanism, one ranking

Pure vector RAG is blind to exact terminology and to the structural relationships between documents (which regulation implements which procedure, which authority is responsible, which procedures are related). Administrative language is dense with exact terms — *Wohnungsgeberbestätigung*, *Tagegeld*, *Vergabevermerk* — where lexical exactness and legal structure matter as much as meaning similarity.

DESIGN CONSEQUENCE

Verwaltungsassistent addresses each weakness structurally: (1) three complementary retrieval mechanisms instead of one; (2) an explicit candidate/evidence distinction; (3) an independent verifier model that judges logical support, not similarity; (4) a fail-closed presentation boundary enforced in code; and (5) a rule-first path that never lets a language model compute what structured knowledge can compute exactly.

3 Verwaltungsassistent Architecture Overview

Verwaltungsassistent is implemented as a modular Java 21 / Spring Boot 3.3 platform. The core reasoning pipeline is domain-agnostic; the municipal application layer supplies domains, terminology, structured rule tables and the user interface. The reference deployment runs entirely on local infrastructure: PostgreSQL for documents, chunks, audit data and user management; Qdrant for vector search; Neo4j for the knowledge graph; and Ollama as the local model server for generation, verification and embeddings.

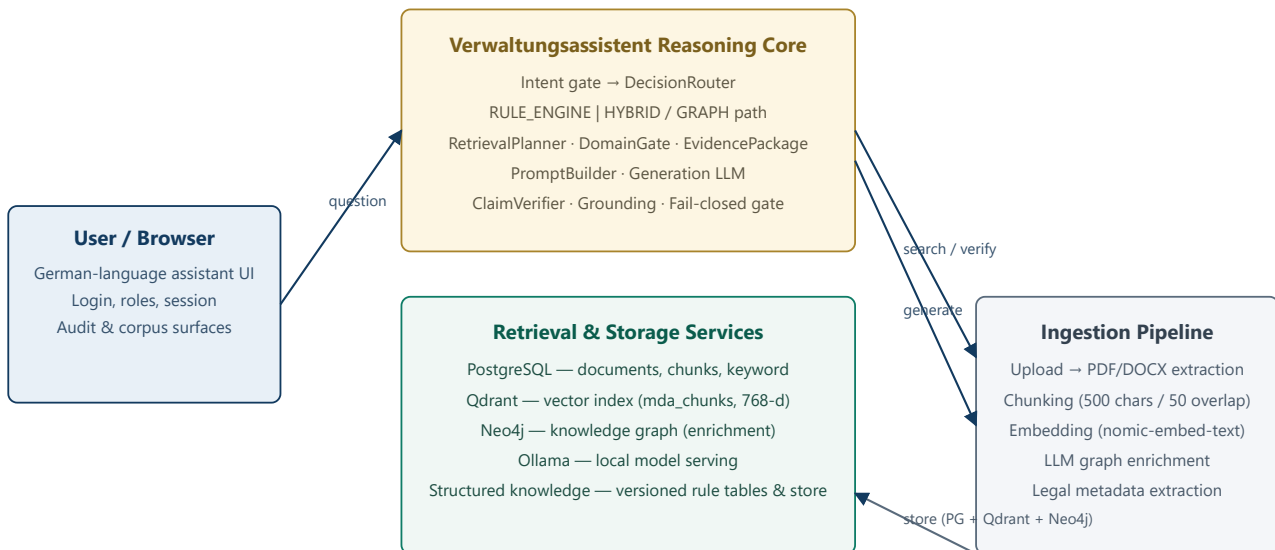


Figure 1 — Verwaltungsassistent system context: the reasoning core orchestrates retrieval, verification and generation; storage services and the ingestion pipeline sit alongside it. All services run locally (on-premise capable).

One execution path answers every question (AiService, single entry point). A query-intent gate first detects meta-queries about the index itself. Then the DecisionRouter routes the question to exactly one strategy:

- **RULE_ENGINE** — the question matches structured knowledge (salary, travel allowance, procurement threshold). A deterministic table lookup produces the decision; the LLM only explains it. Retrieval is structurally skipped.
- **GRAPH_REASONING / HYBRID_RETRIEVAL** — no deterministic match. Full hybrid retrieval runs (graph-enhanced when Neo4j is available), followed by evidence assembly, generation, verification and the fail-closed gate.

Verwaltungsassistent is fed from two channels in the reference deployment: the assistant UI (direct questions) and an **email intake pipeline**. Incoming mail is deduplicated, classified deterministically (request type, whether administrative work is required), matched against existing cases — or opened as a new case (*Vorgang*) — and its attachments are ingested through the same document pipeline. From there, every request runs the identical question-answering path described in this document. Case handling follows explicit phases (SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE); completion requires explicit human sign-off.

RULE-FIRST ARCHITECTURE

If the question has a deterministic structured answer, use structured knowledge. Otherwise, use hybrid retrieval and evidence-grounded reasoning. A language model is never asked to compute a tariff amount or a procurement procedure when a rule table can compute it exactly.

The remainder of this document follows the life of a question and a document through the system: ingestion (Section 4), retrieval (Section 5), intent and domain (Section 6), ranking and evidence selection (Section 7), the evidence problem (Section 8), verification (Section 9), the fail-closed boundary (Section 10), rule-first processing (Section 11), and worked examples (Section 12).

4 Document Ingestion

Before Verwaltungsassistent can answer questions, authoritative documents must be ingested. Ingestion is an asynchronous pipeline: an upload creates a document record and an ingestion job; a scheduled worker processes pending jobs (oldest first, every 10 seconds). Each document passes through extraction, chunking, embedding and three parallel persistence paths.

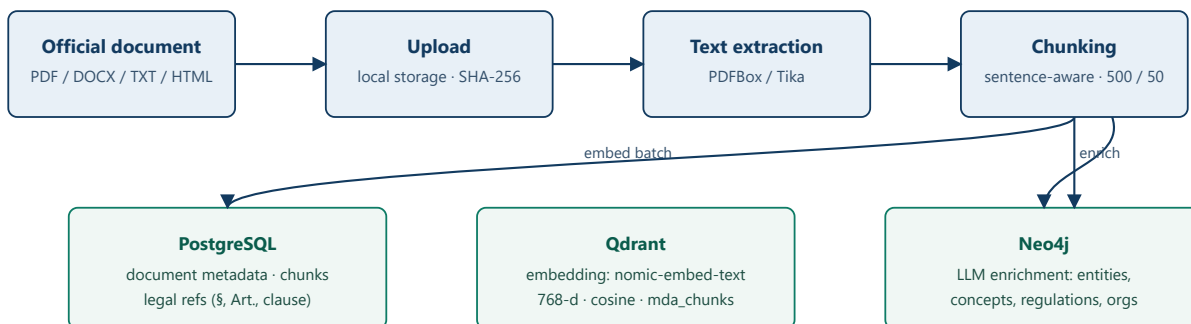


Figure 2 — Ingestion data flow: one document fans out into three storage systems, each serving one retrieval mechanism (lexical, vector, graph).

4.1 Text extraction and chunking

PDFs are extracted with Apache PDFBox (position-preserving); HTML via jsoup; plain text directly; all other formats (DOCX and the like) via Apache Tika. Extracted text is split into chunks by a **sentence-aware strategy**: target size **500 characters with 50 characters overlap**, preferring sentence and paragraph boundaries — including numbered paragraphs and § headings — and falling back to a fixed-size cut only when no boundary is available. Chunk-level metadata includes document title, type, category, tags, and — for administrative documents — extracted legal references (§, Art., clause numbers).

4.2 Three parallel persistence paths

- **PostgreSQL** — document metadata (title, type, status, versions, checksums), ingestion job state, and every chunk with its full text and positions. PostgreSQL also serves as the keyword-search index (Section 5.2). Vectors are *not* stored here; each chunk stores only an embedding reference such as `nomic-embed-text:768d`.
- **Qdrant** — each chunk is embedded with `nomic-embed-text` (768 dimensions) and written as a point into collection `mda_chunks` (cosine distance). The payload carries the chunk text, document identifiers, title, type, category and tags, so search hits are self-describing.
- **Neo4j** — an LLM-based enrichment step extracts entities, concepts, regulations and organizations from the document text and writes a graph: a `DOCUMENT` node, `ENTITY`, `CONCEPT`, `REGULATION` and `ORGANIZATION` nodes, and `MENTIONS / RELATED_TO` edges (among others). The graph gives retrieval a structural view of the corpus: which documents mention which legal bases, which authorities, which related procedures.

4.3 Temporal validity metadata

Documents carry uploader-maintained validity metadata: publication date, valid-from, valid-until and superseded-at. These dates are stored on the document, copied to every chunk and into the vector payload, and travel with each citation. At question time the caller may supply a reference date (*asof* — the German *Stichtag*); retrieval then drops candidates that are not yet valid, already expired or superseded as of that date, while documents with unknown dates are kept and flagged as such. The reference date is also shown to the generator and, through the citation, to the reviewer — so an answer is always anchored to the version of a regulation that applied on the reference date.

Ingestion is idempotent per document version, and the pipeline degrades gracefully: if the embedding service is unavailable, a keyword-only fallback path still indexes chunks into PostgreSQL.

THE DEMO CORPUS (SNAPSHOT 2026-08-15)

The reference deployment runs 19 documents: *Bundesreisekostengesetz (BRKG)*, *Ausführungsvorschriften zu § 55 LHO Berlin* (procurement thresholds), *Bauordnung für Berlin*, and 15 official Berlin Service-Portal procedures (registration/de-registration, identity documents, passport, driver's licence, certifications, eID activation). The ingested corpus currently comprises ~1,560 PostgreSQL chunks, ~2,120 Qdrant points and ~350 graph nodes. The versioned knowledge base additionally contains GWB Part 4 (procurement law), the TV-L 2025 collective agreement and the Berlin Bauvorlagenverordnung as managed markdown documents.

5 Hybrid Retrieval: Vector, Lexical and Graph Search

When the router decides that no deterministic rule answers the question, Verwaltungsassistent executes a **single-pass hybrid retrieval**: three independent search mechanisms run over the same question in parallel, and their results are fused into one candidate pool. Retrieval is deliberately single-pass — no recursive searches, no query rewriting — which keeps the process fast, explainable and easy to audit. Each channel deliberately over-fetches (the vector channel to six times the requested result window, the lexical channel up to a fetch cap of 2,000 chunks) so that fusion, temporal filtering and evidence selection decide on a rich candidate pool rather than on a truncated top list.

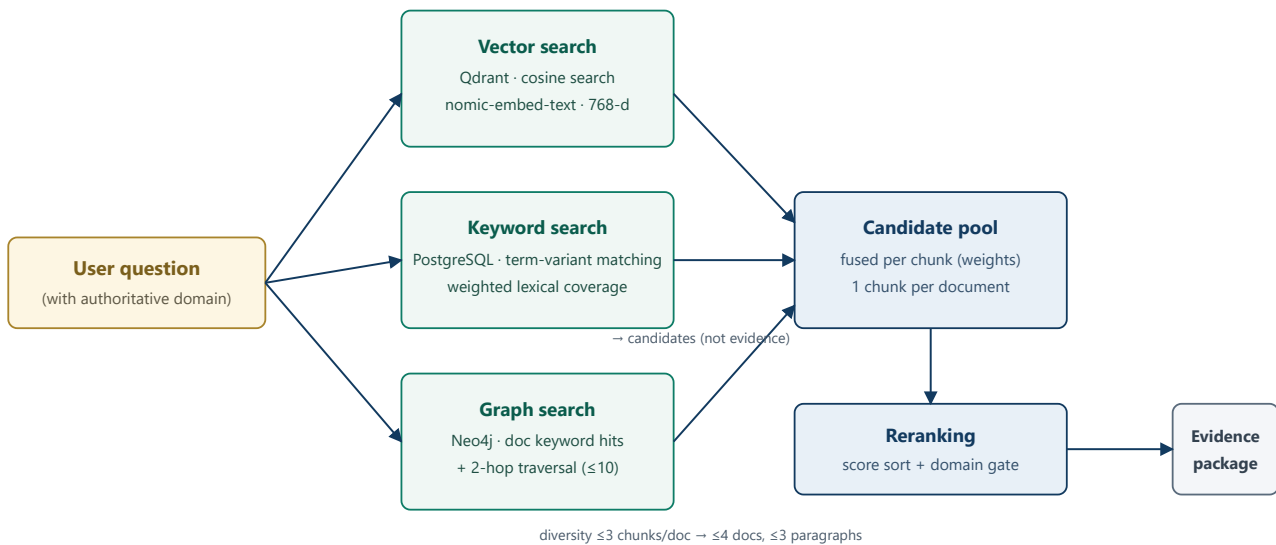


Figure 3 — Hybrid retrieval: three mechanisms, one candidate pool, reranking, then a compact evidence package. Until verification (Section 9), everything in this diagram is a *candidate*, not evidence.

5.1 Vector (semantic) retrieval — Qdrant

An embedding is a numerical vector that places a text in a high-dimensional meaning space: texts with similar meaning land close to each other, regardless of wording. In Verwaltungsassistent, both the question and every ingested chunk are embedded with the local model `nomic-embed-text` (768 dimensions), served through Ollama.

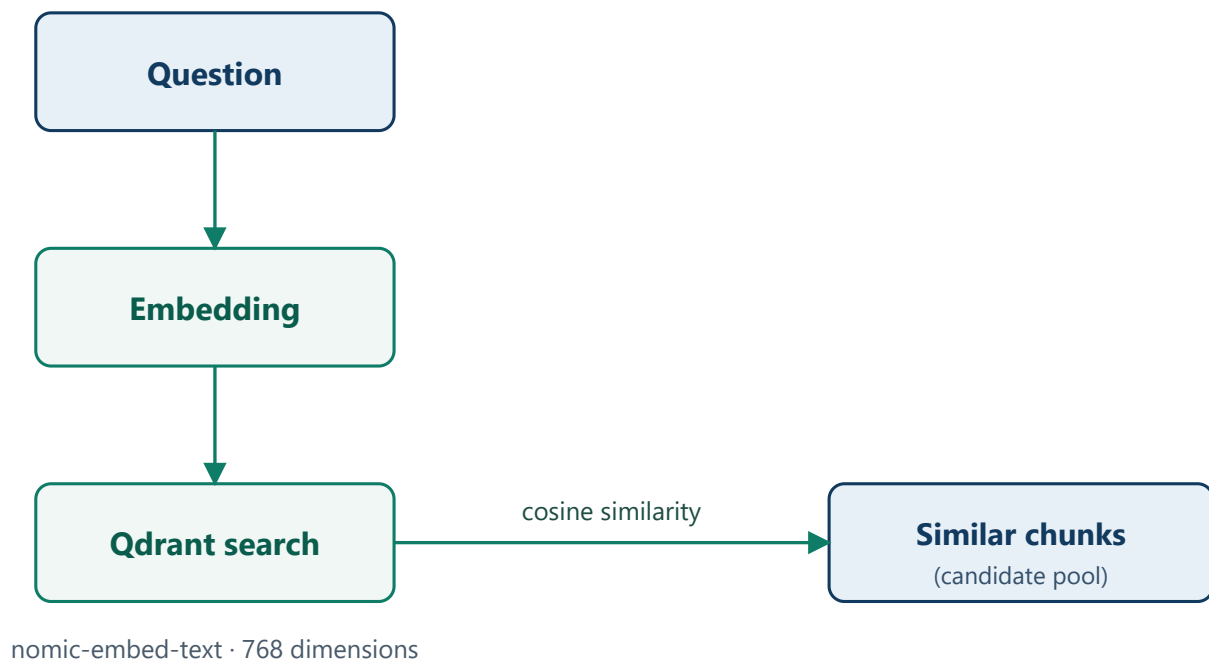


Figure 4 — Vector retrieval: question → embedding → cosine search in Qdrant collection mda_chunks → the most similar chunks as candidates.

What vector retrieval is good at: paraphrase-tolerant matching. A question asking "which documents do I need to apply for an ID card" will find the official procedure "Personalausweis beantragen" even when the wording differs substantially from the document text. It ranks by meaning, not by vocabulary.

What vector retrieval can miss: exact terminology and structural facts. Two passages can be meaning-adjacent yet differ on the one term that decides the answer (a threshold of 1,000 € versus 10,000 €; a requirement versus an exception). Vector similarity also has no notion of which regulation implements which procedure or which authority is responsible — it only sees text.

QDRANT IS ONE COMPONENT OF THE ARCHITECTURE, NOT "THE RAG"

Qdrant provides the vector index and cosine search. It is one of three retrieval mechanisms, and it neither verifies anything nor generates anything. Vector search alone — "Qdrant RAG" — is exactly the pattern this platform is designed to move beyond.

5.2 Lexical (keyword) retrieval — PostgreSQL

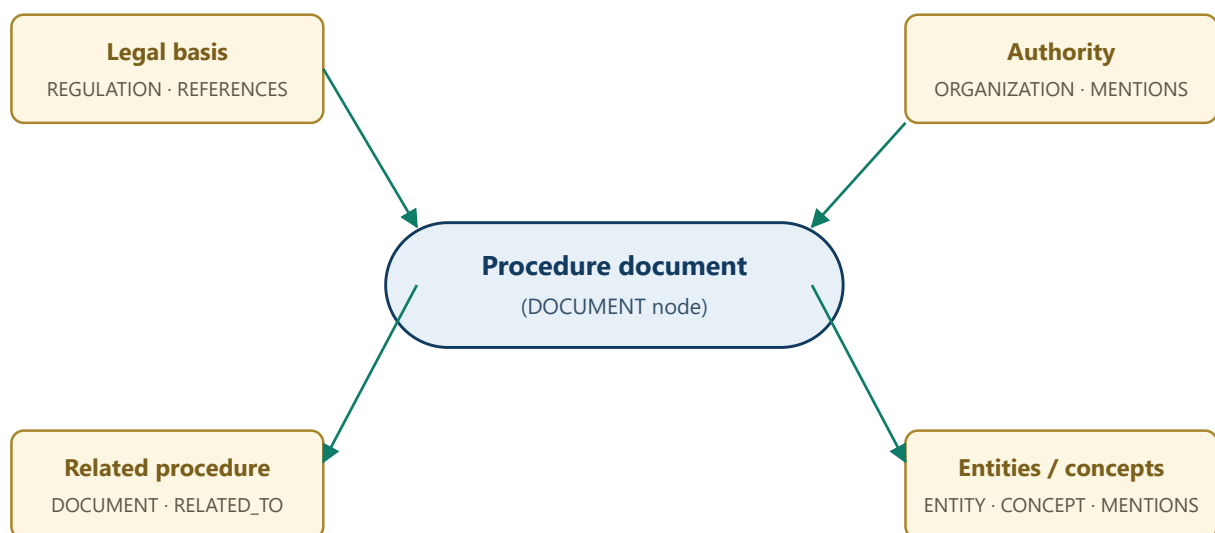
The keyword path matches chunk text in PostgreSQL with case-insensitive pattern matching (`ILIKE`) over the content words of the question and their German term variants: umlaut and β folding ($\ddot{a} \rightarrow a$, $\ddot{o} \rightarrow o$, $\ddot{u} \rightarrow u$, $\beta \rightarrow ss$) plus morphological prefix variants (e.g. *Ummeldung* → *Meldung*). Generic administrative vocabulary is excluded via a stop-word list. Every matching chunk (up to a fetch cap of 2,000) is then scored in the application by **weighted lexical coverage**: the share of the question's word mass — repeated words count more, because they mark the topic — that is actually present in the chunk. Chunks dominated by table-of-contents dot leaders are suppressed, so OCR artifacts cannot masquerade as evidence. This is a deliberately simple, transparent mechanism: the current implementation does not use PostgreSQL full-text search (`tsvector`), BM25 or sparse-vector indexing.

Why keep a lexical path at all when vectors exist? Because administrative text is dense with exact terms whose presence or absence is itself the answer. Consider the term *Wohnungsgeberbestätigung* (landlord's confirmation of residence). A question about this document class retrieves — lexically — the chunks that actually contain the term, wherever they sit in the semantic ranking. Exact terminology is also what regulators, auditors and staff search for; a mechanism that provably matches the literal term is an audit-friendly complement to meaning-based search.

5.3 Graph retrieval — Neo4j

The graph path queries Neo4j in two steps: a keyword match over document nodes (title, label, category, tags; up to 10 documents), then a 2-hop traversal from the matched documents to related nodes — regulations they cite, organizations they involve, concepts they share, and other documents connected to them. Graph results join the candidate pool with a fixed base score (0.6 for document nodes, 0.5 for other node types).

Graph retrieval finds what neither mechanism above can: *documents you did not search for, but which are structurally connected to what you did*. A question about passport applications can surface the related identity-card procedure and the legal bases both procedures cite — material that no similarity measure would rank, but which an administrator needs for a complete answer.



Conceptual illustration — actual node labels are the generic set above (enrichment maps legal bases → REGULATION, authorities → ORGANIZATION)

Figure 5 — Conceptual graph schema. The enrichment stage extracts entities, concepts, regulations and organizations per document; retrieval traverses up to 2 hops from matched documents.

5.4 Why combine all three?

Consider the question: „Eine Person möchte sich anmelden, hat aber keine Wohnungsgeberbestätigung.“ ("A person wants to register, but has no landlord's confirmation.")

- **Vector retrieval** returns semantically related municipal documents — registration, change of residence, deregistration procedures.
- **Keyword retrieval** returns chunks containing the relevant terminology — *Wohnungsgeberbestätigung*, *Anmeldung* — wherever it literally occurs.
- **Graph retrieval** returns related administrative and legal entities — the registration-law bases (REGULATION nodes) and authorities (ORGANIZATION nodes) the registration documents reference.

The three result sets are combined into one candidate pool. Each mechanism compensates for the blind spots of the others; the question is then whether the pool actually *answers the question* — which is decided much later, at verification (Section 9).

6 Intent and Domain Analysis

Before anything is retrieved or computed, Verwaltungsassistent builds a structured understanding of the question. The semantic intent parser extracts a `StructuredIntent`: language (de/en/pt/fr), domain, intent type (`TRAVEL_ALLOWANCE`, `SALARY_LOOKUP`, `PROCUREMENT_THRESHOLD`, or `GENERAL`), and typed parameters (hours, distance in km, amount in EUR, salary grade and step, overnight flag).

The default parser in the reference deployment is an LLM-based parser running at temperature 0 (deterministic decoding) on the local verifier model. Its prompt enforces semantic rules — a number alone is not evidence of intent; amounts require a currency context; hours require a duration context — and language-aware number parsing (e.g. distinguishing thousands separators from decimal separators in German, English, French and Portuguese). A deterministic regex-based parser remains available as a one-flag emergency fallback, preserving a no-LLM path for intent extraction.

6.1 Domains

The reference deployment registers four administrative domains — `PROCUREMENT`, `BUILDING`, `HR`, `TRAVEL` — plus the neutral `GENERAL` domain. Domain terminology (weighted terms per domain) is externalized in a YAML knowledge file, so domains can be adapted to a new authority without code changes. A weighted keyword classifier (`DomainClassifier`) exists as the legacy/fallback domain signal; when the LLM parser supplies a domain, that domain is *authoritative* and is propagated through the entire pipeline — into the retrieval plan and into the domain-aware ranking stage.

DOMAIN-COHERENCE GATE

A deterministic rule may execute only if the intent carries a domain matching the domain its rule family requires (e.g. `TRAVEL_ALLOWANCE` requires domain `TRAVEL`). An isolated parameter — even a plausible one — never triggers a rule without a coherent domain. This prevents semantically similar but domain-irrelevant interpretations from hijacking the deterministic path.

6.2 How domain information shapes ranking

Domain influences candidate ranking through a *soft* gate, not a hard filter (`DomainGate`). After hybrid fusion and reranking, each candidate's score is multiplied by a domain relevance factor derived from its document title: documents matching the query's domain (or domain-neutral documents) keep their score (factor 1.0); documents that clearly belong to a *different* domain receive a mild penalty (factor 0.8) and are re-sorted accordingly. Cross-domain documents are thus demoted, not deleted — a hard filter could drop a genuinely relevant cross-referenced regulation. This mechanism is what keeps, for example, travel-expense regulations out of the top ranks of a procurement question, without silently censoring the result set.

7 Candidate Ranking: Fusion, Reranking and Evidence Selection

Retrieval produces *candidates* — never guaranteed evidence. Three ranking steps transform the raw pool into the compact evidence package that the generator sees.

7.1 Fusion of the three result sets

Fusion happens at chunk level: when a chunk was found by more than one mechanism, its signals are combined. The fused ranking score is a weighted combination of the keyword score, the vector score and a confidence component, multiplied by a document-type weight derived from query intent (the weights are configuration; the reference deployment uses 0.20 / 0.60 / 0.20):

$$\text{rankingScore} = (\text{0.20} \cdot \text{keywordScore} + \text{0.60} \cdot \text{vectorScore} + \text{0.20} \cdot \text{confidenceScore}) \cdot \text{documentTypeWeight}$$

Graph hits add a modest additive boost to an existing candidate (15% of the graph score, capped at 1.0) rather than competing as a third proportional weight. This reflects the role of the graph as structural evidence for documents already found by the text-level mechanisms. The pool is then deduplicated to one record chunk per document: among chunks that literally carry question terminology, the best fused score wins; only if a document has no lexically anchored chunk does its overall best chunk represent it. After deduplication, a lexical anchor rule drops candidates with no overlap with the question at all (skipped when it would leave fewer than two candidates), a relative semantic floor (half of the best vector score, at least 0.20) removes weakly related tail candidates, and sister-chunk expansion re-attaches adjacent passages (a 500-character window around each selected chunk) so split sentences remain readable in the evidence package.

WHAT THE IMPLEMENTATION IS — AND IS NOT

The current implementation uses the fixed weighted fusion above with an intent-based document-type multiplier, plus the additive graph boost. It does *not* use reciprocal-rank fusion, learned weights, or a naive 40/30/30 split across the three mechanisms.

7.2 Reranking

The reranking stage orders the fused candidates by ranking score. The platform also contains an LLM-based cross-reranker (a local model scoring each candidate's relevance to the question, blended 80/20 with the retrieval score) — implemented, but *not enabled* in the current reference configuration; the active reranking is the deterministic score sort followed by the domain gate.

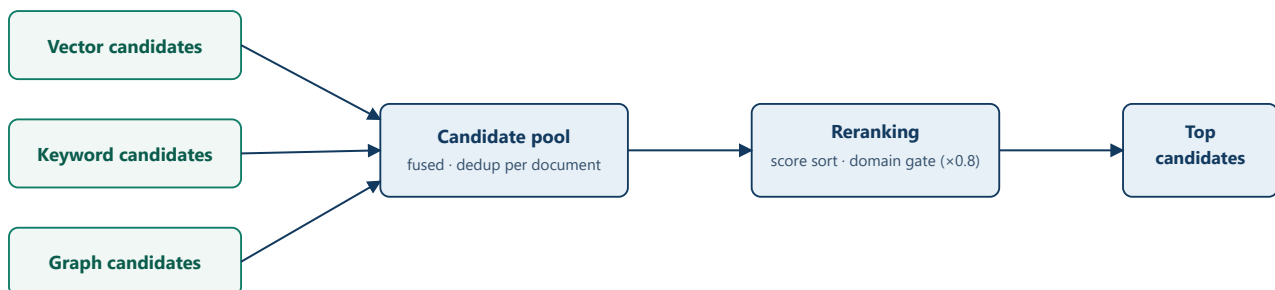


Figure 6 — Candidate ranking: three retrieval streams → one fused pool → reranking with domain adjustment → top candidates. Still candidates: verification has not happened.

7.3 Evidence selection: the evidence package

The final compression step (`EvidencePackageBuilder`) converts ranked candidates into the small, high-quality package the generator may use:

- **Source cap** — at most 4 documents per question.
- **Diversity cap** — at most 3 excerpts (paragraphs) per document.
- **Excerpt limit** — each excerpt truncated to 1,200 characters.
- **Lexical relevance floor** — a document must cover at least 30% of the question's word mass; from the second document on, a relative floor applies as well ($\max(0.20, \text{half of the best document's coverage})$), so weak tail documents fall away instead of diluting the prompt.
- **Sufficiency label** — the package counts as *sufficient* only when at least two documents are used and the leading document's confidence reaches 0.6; a single strong document is labelled *partial*, not sufficient.

This discipline is deliberate: a focused prompt with 3–4 documents is far less likely to produce unsupported claims than a 15-candidate dump. The total prompt stays below 3,800 characters, which also bounds generation latency and cost.

8 The Critical Evidence Problem

SEMANTIC SIMILARITY $\neq$ EVIDENTIARY SUPPORT

A document can be highly similar to a question and still not contain the information necessary to answer it. Retrieval ranks relatedness; only verification can establish support. In Verwaltungsassistent every retrieved item is a *candidate* until the verification stage confirms it as *evidence* for the specific claims of the specific answer.

The clearest demonstration comes from the reference deployment's own development history. Consider:

„Eine Person will sich anmelden, hat aber keine Wohnungsgeberbestätigung.“
"A person wants to register (their residence), but has no landlord's confirmation of residence."

Retrieval for this question works *excellently* in the naive sense: the demo corpus contains 15 Berlin registration-related procedure documents, and vector, keyword and graph search all return registration material — *Anmeldung*, change of residence, deregistration, their legal bases and authorities. Every one of these documents is genuinely related to Berlin administration, and several are genuinely about residence registration.

But none of them contains the specific piece of information the question asks for: the exception case for registration *without* a Wohnungsgeberbestätigung. The retrieved documents describe the standard procedure, in which the confirmation is required. Top-k similarity — even near-perfect topical fit — does not answer the question. A naive RAG system would still generate an answer: the model would plausibly describe the standard registration procedure, possibly adding a general caveat from its own knowledge, cite the retrieved documents, and present the result confidently. The answer would look grounded and would be wrong in exactly the way that matters.

RETRIEVED CANDIDATE $\neq$ VERIFIED EVIDENCE

This distinction is the fundamental difference between Verwaltungsassistent and simplistic top-k RAG. Candidates are inputs to verification, not outputs to the user. The user never sees "retrieved documents" presented as proof; the user sees *verified citations* — and only if verification succeeded.

9 Evidence Verification

After generation, Verwaltungsassistent runs an **independent verification stage**. The generated answer is decomposed into claims (its sections: KURZANTWORT (short answer), ENTSCHEIDUNG (decision), RECHTSGRUNDLAGE (legal basis), VERFAHREN (procedure), NÄCHSTER SCHRITT (next step)); each claim is then judged against the retrieved evidence excerpts by a *separate* verifier model — a different model instance and different LLM calls from the generator.

9.1 What the verifier receives

- The extracted claims of the generated answer (each truncated to 400 characters).
- The evidence excerpts from the evidence package (each truncated to 500 characters for the verifier), identified as E1...En — at most the five highest-confidence evidence items are submitted.
- In the default batched strategy, additionally the **user question itself**, together with "critical rules" — most importantly: *"Retrieved candidates are NOT automatically evidence. A high retrieval similarity score does NOT mean a candidate is relevant to the USER QUESTION."* and *"If the available evidence is insufficient ..., the correct result is an insufficient-evidence outcome (UNKNOWN for all claims) — never an inferred or invented answer."*

9.2 Verdicts

For every claim/evidence relationship the verifier returns one of three verdicts with a confidence and a reason:

Verdict	Meaning	Consequence for the claim
ENTAILED	The evidence logically supports the claim; the claim follows from the evidence.	Supported finding (primary). The claim may be presented, citing this evidence.
CONTRADICTED	The evidence contradicts the claim (negation, different numbers, opposite obligation).	Contradicted finding (secondary). Counts against grounding.
UNKNOWN	The evidence is insufficient to determine; the claim concerns different facts.	Unsupported finding (secondary). Counts against grounding.

The verifier prompt is explicit about the traps that make this judgment hard: negation ("not required" vs "required" is a contradiction, not a near-miss), differing numerical values ("12 EUR" vs "24 EUR" contradict when the evidence fixes the value), obligation strength ("must" vs "may"), exceptions, temporal qualifications and paraphrases.

9.3 Why an independent verifier — and what it cannot do

The generator is asked to write a plausible answer; a model asked to check its own work inherits its own blind spots. The verifier's task is narrower — judgment of logical entailment between a fixed claim and a fixed excerpt — and it runs as separate calls with its own model configuration. In the reference deployment the verifier runs on qwen2.5:14b; the validated qwen2.5:7b verifier remains a one-variable rollback option, and both were evaluated on a controlled entailment benchmark (see Section 13).

The verifier is *not infallible*: it is an LLM, and its judgments are probabilistic. Its verdicts are recorded in the audit trail with reasons, and its failures degrade safely — a verification error yields UNKNOWN, which counts against grounding, never for it.

9.4 Verification strategies (configurable)

- `claim_batch` (default) — one verifier call covering all claims and all evidence items of the question, including the user question and the critical rules above. Reduces verifier round-trips by roughly an order of magnitude while keeping the independent-verifier role.
- `pairwise` — one verifier call per claim/evidence pair. Maximally isolated judgments; the historical baseline and the one-flag fallback.

9.5 How verification shapes the final result

Verification results feed the grounding decision, but the final present/withhold status is deliberately **deterministic**: it is computed from stable artifacts only — evidence whose excerpt or document title carries a specific term of the question (the *anchor rule*), and the lexical support of the drafted core answer against that anchored evidence. Stochastic per-claim verdicts inform the findings and the audit trail (a verified contradiction is treated as a blocker) but cannot flip the status from run to run for the same question, corpus and retrieval. Citations are re-scored against the final answer text (lexical overlap boost) and tiered into PRIMARY / SUPPORTING / BACKGROUND, so the UI can distinguish the sources that actually carry the answer from peripheral material.

10 Fail-Closed Answer Boundary

The verification stage is not advisory. It is the **authoritative safety boundary** of the system, enforced in code by a deterministic gate: an answer is presented only when the evidence package contains at least one item *anchored* to the question — an excerpt or document title carrying a specific term of the question — and the drafted core answer is lexically supported by that anchored evidence. Otherwise the generated answer is **not presented**: the user receives the system's explicit insufficient-knowledge response, and the citations are cleared:

„Für diese Frage liegen in der Wissensbasis keine ausreichenden Informationen vor.“
 "The knowledge base does not contain sufficient information for this question."

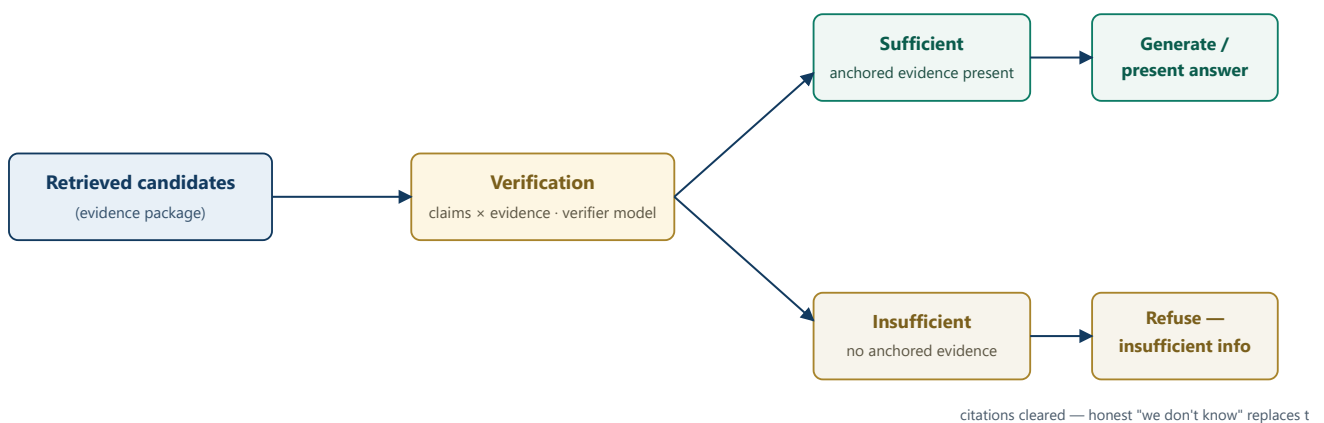


Figure 7 — The fail-closed boundary: verification decides between presenting the grounded answer and replacing it with the insufficient-information response.

The boundary also holds in the retrieval-failure direction: if retrieval produces no usable sources at all, the same insufficient-information response is returned, and the generation prompt itself contains an explicit no-documents block („KEINE DOKUMENTE GEFUNDEN.“ — "NO DOCUMENTS FOUND") instructing the model that the knowledge base contains no sufficient information. The unsupported and contradicted findings remain recorded as the honest verification trail.

RETRIEVAL FAILURE MUST NOT BECOME FABRICATED CERTAINTY

Fail-closed means: when the evidence does not support an answer, the system's answer is "insufficient information" — not the model's best guess. For an administrative assistant this is the single most important behavioral guarantee: users learn that a confident answer means verified support, and that silence means the knowledge base has a gap — which the platform surfaces as a corpus-coverage finding rather than hiding.

11 Rule-First Processing and Structured Knowledge

Many high-value administrative questions are *computable*: they have exactly one correct answer given the applicable table. For these, Verwaltungsassistent never asks a language model to reason. The router looks up the answer in structured knowledge and the LLM is used only to *explain* the computed decision — in the user's language, in a fixed format, with the legal basis attached. Retrieval is structurally skipped on this path.

11.1 Structured knowledge tables in the reference deployment

The municipal application registers three authoritative tables at startup (versioned, reloadable registry). The core platform is generic: it iterates whatever tables an application registers — the tables themselves are municipal data, not platform code.

Table	Source	Effective	Representative values
TV-L salary (Entgelttabellen)	TV-L 2025 collective agreement	2025-02-01	EG 1–15 × Stufen 1–6, e.g. EG 5 Stufe 2 = 3,362.72 €; EG 9b Stufe 3 = 4,117.53 €
BRKG travel allowances (Reisekosten)	Bundesreisekostengesetz (BRKG)	2024-01-01	Tagegeld 24 h = 24 € · 11–24 h = 12 € · 8–11 h = 6 € · arrival/departure day = 12 € · mileage 0.35 €/km (car), 0.20 €/km (motorcycle) · accommodation 80 € (with receipt) / 20 € (flat rate)
Procurement thresholds (Vergabeverfahren)	Ausführungsvorschriften zu § 55 LHO Berlin	2024-01-01	<500 € no formal procedure · 500–1,000 € direct award with approval · 1,000–10,000 € direct award with procurement note · 10,000–100,000 € restricted tender (3 comparison quotes) · ≥100,000 € public/EU-wide tender · construction ≤20,000 € direct award

Beyond these registered tables, the platform ships a **generic structured-knowledge store**: items of kind TABLE, THRESHOLD, RULE, DEFINITION, EXCEPTION OR EFFECTIVE_DATE, with the lifecycle CANDIDATE → VALIDATED → ACTIVE (or rejected / superseded). Items can be extracted from ingested documents by an LLM worker; before an item may become ACTIVE, a validator requires provenance and numeric support and a successful entailment check through the same claim-batch verification machinery used for answers. The extraction worker is *disabled by default* in the shipped configuration: automated knowledge growth is an explicit operator decision, not a silent side effect.

11.2 The deterministic path

For a question such as *"Wir beschaffen Büromaterial für 3,500 € — welches Vergabeverfahren?"*, the pipeline is: intent parse (PROCUREMENT_THRESHOLD, amount = 3,500 €) → domain-coherence gate (domain PROCUREMENT required and present) → threshold table lookup (1,000–10,000 € → *Direktauftrag* with *Vergabevermerk*) → deterministic decision at confidence 0.98 → LLM explanation prompt containing the computed decision, the applied threshold, the legal basis (AV § 55 LHO Berlin) and the responsible authority. The generation prompt explicitly forbids the model from choosing a different procedure or performing its own calculations. No retrieval, no verification against documents — the table is the authority, and the answer is presented as grounded with the legal basis as primary authority reference.

Two further guards protect this path: a salary lookup requires *both* grade and step in the question (no silent default for a missing step — the question falls through to retrieval instead of inventing one), and each parameter class is authorized only by its matching intent type (an isolated "12 hours" in a GENERAL question never triggers a travel rule).

11.3 Two answer classes, clearly separated

	Deterministic rule answer	Retrieval-grounded generative answer
Trigger	Structured knowledge resolves the question exactly	No deterministic match — hybrid retrieval runs
Computation	Rule table lookup (exact)	LLM reasoning over verified evidence only
LLM role	Explain-only; forbidden to alter the decision	Generate; restricted to supplied evidence
Verification	Not needed — table is authoritative	Mandatory claim verification + fail-closed gate
Citations	Legal basis as primary authority reference	Verified excerpts with page numbers, tiered

12 End-to-End Examples

Three worked examples follow the actual pipeline. Question texts are German, as used by the reference deployment; English translations are given in parentheses.

Example 1 — Deterministic rule: travel allowance

„Wie hoch ist das Tagegeld bei einer Dienstreise von 12 Stunden?“

"What is the daily allowance for a 12-hour business trip?"

question → semantic intent parse → routing → structured lookup → rule answer

1. **Intent:** the LLM parser returns TRAVEL_ALLOWANCE, domain TRAVEL, hours = 12, language de.
2. **Domain-coherence gate:** TRAVEL_ALLOWANCE requires domain TRAVEL — coherent, lookup proceeds.
3. **Structured lookup:** the BRKG table resolves 12 h to the 11–24 h bracket: *Tagegeld 12,00 €*. Decision confidence 0.99.
4. **Deterministic path:** no retrieval. The explanation prompt carries the computed decision ("Tagegeld: 12 € (11-24h)"), the legal basis *Bundesreisekostengesetz (BRKG)* and the authority; the LLM formats it as KURZANTWORT / ENTSCHEIDUNG / RECHTSGRUNDLAGE. The answer is presented as grounded with the BRKG as primary authority reference.

Example 2 — Deterministic rule: procurement threshold

„Wir möchten Büromaterial im Wert von 3.500 € beschaffen. Welches Vergabeverfahren ist anzuwenden?“

"We want to procure office supplies worth 3,500 €. Which award procedure applies?"

1. **Intent:** PROCUREMENT_THRESHOLD, domain PROCUREMENT, amountEur = 3500 (thousands separator parsed correctly as 3500, not 3.5).
2. **Gate:** domain coherent → threshold lookup authorized.
3. **Lookup:** AV § 55 LHO Berlin, Lieferung/Dienstleistung, 1,000–10,000 € → *Direktauftrag*, requirement: *Vergabevermerk* (procurement note). Confidence 0.98.
4. **Presentation:** deterministic decision + legal basis (AV zu § 55 LHO Berlin) + authority, explained by the LLM in the standard format. No retrieval, no document verification — the rule table is the authoritative source.

Example 3 — Successful retrieval: applying for an identity card

„Welche Unterlagen benötige ich, um einen Personalausweis zu beantragen?“

"Which documents do I need to apply for an identity card?"

1. **Routing:** no structured table resolves this (intent GENERAL) → retrieval path, graph-enhanced hybrid mode (Neo4j available).
2. **Hybrid retrieval:** vector search finds the official procedure *ServicePortal-120703 — Personalausweis beantragen* (semantically closest to the question, in the demo corpus); keyword search confirms chunks literally containing the required-documents terminology; graph search adds related identity procedures (e.g. passport, provisional identity card) and the legal bases they cite.
3. **Fusion and ranking:** the three streams merge into one candidate per document; domain adjustment is neutral (question domain GENERAL); the evidence package selects ≤4 documents × ≤3 excerpts of up to

1,200 characters — the identity-card procedure dominating.

4. **Generation:** the prompt contains only those evidence blocks plus the strict rules ("use only the evidence, invent no regulations, quote numbers exactly"). The model drafts the answer in the standard section format, citing documents.
5. **Verification:** the excerpts are anchored to the question (they carry its specific terms) and the drafted core answer is lexically supported by them — the required-documents claims (biometric photo, previous identity document, registration certificate) are corroborated and nothing is contradicted. The answer is presented as sourced, citations are tiered, and the user sees the answer with links to the source document, page references and excerpts.

Example 4 — Unsupported retrieval: registration without a landlord's confirmation

„Eine Person will sich anmelden, hat aber keine Wohnungsgeberbestätigung.“

"A person wants to register, but has no landlord's confirmation."

1. **Routing:** no rule table matches → retrieval path.
2. **Retrieval:** the demo corpus is rich in registration material; vector, keyword and graph search all return related candidates — registration, change of residence, deregistration procedures, their legal bases and authorities. Top-k retrieval looks highly successful.
3. **The evidence problem:** none of the retrieved documents states the exception case the question asks about (registration *without* the confirmation). The candidates are about Berlin administration — and about registration — but they are not evidence *for this question*.
4. **Verification rejects the unsupported claims:** the retrieved excerpts concern the standard procedure — they do not carry the specific terms of the requested exception, so no anchored evidence exists and the drafted claims find no lexical support.
5. **Fail-closed:** the draft answer is discarded. The user receives *„Für diese Frage liegen in der Wissensbasis keine ausreichenden Informationen vor.“* with citations cleared. The unsupported findings remain in the audit trail, flagging a genuine knowledge-base gap (the exception document is not yet in the corpus) — actionable for corpus management.

EXAMPLE 4 IS A SAFETY FEATURE, NOT A FAILURE

A top-k RAG system would have produced a confident, cited, plausible — and wrong — answer here. Verwaltungsassistent's refusal is the designed outcome of the candidate/evidence distinction and the fail-closed boundary. In the reference deployment this exact case drove the introduction of the verification gate.

13 Performance Considerations

End-to-end latency is dominated by LLM inference, not by retrieval. Each question triggers at least one LLM call (intent parsing), one generation call, and one or more verification calls; retrieval itself consists of one SQL keyword query, one vector search round-trip and one graph query — each typically measured in tens to hundreds of milliseconds on the demo host.

Stage	Typical role	Relative cost
Intent parsing	LLM call, temperature 0, short output (JSON)	Low–medium (one small LLM call)
Keyword retrieval	Term-variant pattern matching over chunk text, scored in-process (PostgreSQL)	Low (database query + in-memory scoring)
Vector retrieval	One embedding + cosine search in Qdrant (over-fetched candidate depth)	Low (one embedding + index search)
Graph retrieval	Keyword match + 2-hop traversal (Neo4j)	Low (graph query)
Fusion / reranking / evidence package	In-memory arithmetic and sorting	Negligible
Answer generation	Generation model (14B local), ≤4k-char prompt	High (dominant single call)
Evidence verification	Verifier model: one batched call per question (claim_batch, default) or one call per claim/evidence pair	High (batched: constant; pairwise: scales with claims × evidence)

DEVELOPMENT MEASUREMENTS — NOT PRODUCTION GUARANTEES

In a controlled verification experiment on the demo host (single NVIDIA Quadro P5000, 16 GB VRAM, local Ollama serving, August 2026), the qwen2.5:7b verifier completed a verification batch in ≈ 2.2 s with 100% correct verdicts on the entailment benchmark, while qwen2.5:14b took ≈ 4.8 s with 96.3% correct verdicts. Both models were benchmarked because both are supported configurations; the 14B model is the current default. These figures are development-environment measurements for sizing intuition only and must not be read as production SLAs; production sizing depends on GPU class, batch load and model choice, and the platform's model endpoints are fully configurable.

Because verification cost scales with the number of claims and evidence items, the evidence package is deliberately small (≤ 4 documents, ≤ 3 paragraphs each, ≤ 5 claim sections), and the default `claim_batch` strategy collapses verification into a single call per question. Retrieval, by contrast, is cheap — which is why Verwaltungsassistent can afford three retrieval mechanisms where a single vector search would cost marginally less but miss materially more.

14 Auditability and Explainability

Administrative use requires that every answer be reconstructable. Verwaltungsassistent records the evidence trail at every stage, into a dedicated audit store (PostgreSQL `audit_events`):

- **Retrieval events** — per question: search mode, classified intent, candidate counts per mechanism (keyword / vector / graph), merged result count.
- **Inference events** — per question: chosen strategy (`RULE_ENGINE` / `GRAPH_REASONING` / `HYBRID_RETRIEVAL`), whether retrieval executed, model used, overall confidence.
- **Execution trace** — per-stage timing (intent, routing, retrieval, prompt, LLM, coverage, grounding) and per-stage logs.
- **Verification trail** — per claim: verdict, confidence, verifier reason, supporting evidence references; contradicted and unsupported findings are retained even when the answer is withheld.
- **User-facing explanation** — every presented answer carries its citations (document, page, excerpt) tiered `PRIMARY` / `SUPPORTING` / `BACKGROUND`, its legal-basis references, and an overall confidence profile; the UI marks answers as grounded ("durch Quellen belegt") or as having limited sourcing, and audit-log views (with CSV export) are available to `AUDITOR`/`ADMIN` roles.

Retrieval provenance is preserved per candidate: the system records *which* mechanism discovered each cited chunk (`KEYWORD` / `VECTOR` / `GRAPH`, possibly several), so an auditor can see not only what was cited but how it was found.

Answers produced in the case-workflow path are additionally stored as **versioned analysis runs**: each run records its status, timestamps and the identity of the evidence it used (document id plus version). A later review can therefore tell whether the evidence base has changed since an answer was produced — replaced or removed evidence is detectable, not silently overwritten.

15 Security, On-Premise and Air-Gapped Deployment

Verwaltungsassistent is designed for authorities that cannot send administrative questions or document corpora to external clouds. The entire runtime — PostgreSQL, Qdrant, Neo4j and the model server — runs on local infrastructure; in the packaged VM deployment only the web port and SSH are exposed, while the database and model ports remain VM-internal. All models (generation, verification, embeddings) run locally via Ollama; no question, document or answer leaves the deployment. Runtime operation is air-gapped capable; only initial model downloads require internet access.

Access control for the reference deployment: Spring Security form login with BCrypt (cost 12) password hashing, CSRF protection, a managed session registry (bounded concurrent sessions, forced re-login on expiry), and role-based authorization on every surface — assistant, decisions, audit log, corpus management, administration (ADMIN), profile. Login, logout and administrative actions are themselves audit events. The platform's API application uses the same user store with self-issued short-lived JWTs (access 15 min / refresh 30 days) and refresh-token rotation. A hardened, non-root container build and a reproducible VM image bundle complete the deployment story (see References).

Untrusted content boundary: ingested documents, email bodies and attachments are data, never instructions. Every prompt that carries retrieved content prefixes it with an explicit boundary marker stating that the following material comes from external, untrusted sources and must not be followed as instructions — a structural defense against prompt-injection attempts hidden in ingested documents.

16 Comparison with Conventional RAG

The table compares architectural patterns, not specific vendor products. "Conventional vector RAG" denotes the common pattern of embedding-based retrieval feeding a generator, without independent verification.

Capability	Conventional vector RAG	Verwaltungsassistent
Semantic (vector) retrieval	Yes	Yes (Qdrant, local embeddings)
Lexical / keyword retrieval	Varies (often absent)	Yes (PostgreSQL term-variant matching, weighted lexical-coverage scoring)
Knowledge-graph retrieval	Usually no	Yes (Neo4j, 2-hop traversal over enriched corpus)
Fusion of retrieval signals	Usually single mechanism	Weighted fusion per chunk + graph boost + document dedup
Domain-aware ranking	Usually limited	Yes — authoritative intent domain, soft cross-domain penalty
Candidate/evidence distinction	Often weak — retrieved = cited	Explicit: retrieved is candidate; verified is evidence
Independent verification	Often absent	Separate verifier model; per-claim ENTAILED / CONTRADICTED / UNKNOWN
Fail-closed on unsupported answers	Often absent	Enforced in code — explicit insufficient-information answer, citations cleared
Deterministic rule engine	No	Rule-first: salary, travel, procurement tables answered exactly, LLM explains only
Multi-language intent handling	Varies	Yes — de/en/pt/fr intent parsing, language-aware number formats
Auditability	Varies	Per-stage audit events, verification trail, retrieval provenance, CSV export
Deployment model	Often cloud-dependent	Fully local: on-premise / air-gapped capable, local models

The differences are structural, not cosmetic: verification and the fail-closed gate are separate pipeline stages with their own model and their own authority to suppress an answer; the rule path removes the language model from computation entirely for the question classes where wrong arithmetic is unacceptable.

17 Limitations

Verwaltungsassistent is designed for trustworthiness, not omniscience. Its limitations are known and deliberately surfaced:

- **Retrieval can still miss relevant information.** Single-pass retrieval without query rewriting is fast and auditable but can miss what a multi-round strategy might find; the evidence package also truncates (≤ 4 documents) for quality, at the cost of completeness.
- **Lexical search is deliberately simple.** The current keyword mechanism is term-variant pattern matching with deterministic coverage scoring — no full-text/BM25 ranking yet. Very large corpora or heavily inflected languages would motivate upgrading this component.
- **Graph completeness depends on ingestion enrichment.** The graph is only as good as the LLM-based enrichment that builds it; un-ingested or poorly enriched documents are invisible to graph retrieval.
- **LLM verification is probabilistic.** The verifier can err; misjudged verdicts flow into grounding. The design minimizes harm (errors default to UNKNOWN, which fails closed, and the trail is auditable), but verification is not formal proof.
- **Model latency.** Local 14B-class generation and verification cost seconds per call; interactive latency depends on available GPU capacity. The batched verification strategy (the default) bounds this per question.
- **Corpus coverage.** The system can only know what was ingested. The insufficient-information answer is honest, but its frequency is a function of corpus maintenance.
- **Regulations change.** Structured tables (BRKG, TV-L, AV § 55 LHO) are versioned data with effective dates; keeping them current is an operational duty, and superseded tables require explicit replacement.
- **Current stage of maturity.** The reference deployment is a validated pilot/demo system (single municipal corpus, demo-scale infrastructure, one active application). It is not yet proven at multi-tenant or large-corpus production scale.

WHY THE LIMITATIONS LIST EXISTS

Every one of these limitations is visible to operators: retrieval misses appear as corpus-coverage gaps, verification errors as UNKNOWN verdicts in the audit trail, rule-table changes as versioned data. A system that hides its failure modes cannot be trusted; Verwaltungsassistent's failure modes are first-class, observable outcomes.

18 Future Directions

Everything in this section is **future work**. None of it is part of the current implementation described in Sections 3–16.

- **Customer-specific model adaptation** — PEFT/LoRA-adapted models per authority or domain, fine-tuned on the customer's own corpus.
- **Additional document sources and automated ingestion** — connectors for document management systems, scheduled re-ingestion of changed regulations, automated corpus monitoring.
- **Workflow integration** — connecting the case workflow to external document-management and workflow systems.
- **Operational knowledge extraction** — the generic structured-knowledge extraction worker ships disabled; enabling it with operator review, and broadening domain-specific structured knowledge (fees, deadlines, entitlements), is future work.
- **Retrieval upgrades** — full-text/BM25 lexical search, multi-round retrieval and query rewriting, activation of the implemented LLM cross-reranker, per-domain collections and authority filters (the retrieval plan already carries these fields; they are currently unused).
- **Improved model serving** — batched inference serving, model quantization options, GPU scheduling for multi-tenant deployments.
- **Scale-out** — multi-tenant operation, larger corpora, production-grade operations (monitoring, backup, disaster recovery) beyond the current pilot setup.

19 Conclusion

The central architectural story of Verwaltungsassistent is the disciplined path from question to answer:

QUESTION → understand intent/domain → retrieve candidates through multiple complementary mechanisms
→ combine and rerank candidates → determine whether candidates actually support the question
→ verify evidence → only then generate and present an answer

Two commitments make this more than a vector RAG system. First, **RETRIEVED ≠ VERIFIED**: the system never confuses what it found with what it proved. Second, **if the evidence is insufficient, Verwaltungsassistent says so** — an explicit, first-class answer, enforced in code, with the unsupported draft discarded and the gap recorded. Combined with rule-first deterministic answers for the question classes that admit exact computation, Verwaltungsassistent offers public administration something rare in generative AI deployments: a boundary between confident answers and honest refusal, drawn systematically, auditably, and on-premise.

Glossary

RAG (Retrieval-Augmented Generation)

A pattern in which retrieved text passages are supplied to a language model as context for generation. Verwaltungsassistent implements a verification-augmented variant of this pattern.

Embedding

A numerical vector representing the meaning of a text. In Verwaltungsassistent, chunks and questions are embedded with the local `nomic-embed-text` model (768 dimensions).

Vector search

Search by embedding proximity (cosine distance) — finds semantically similar text regardless of wording. In Verwaltungsassistent, served by Qdrant.

Lexical retrieval

Search by literal term matching. In Verwaltungsassistent, case-insensitive pattern matching over chunk text in PostgreSQL using German term variants, scored deterministically by weighted lexical coverage.

Knowledge graph

A network of entities and relationships. In Verwaltungsassistent, Neo4j holds enriched document structure: documents, entities, concepts, regulations, organizations and their relationships.

Reranking

Re-ordering of fused retrieval candidates by ranking score (optionally with domain adjustment) to produce the final candidate order.

Candidate evidence

Material retrieved because it looked relevant. Not yet verified; input to verification, never presented as proof.

Verified evidence / grounding

Evidence that actually supports the answer. An answer is presented only when the evidence package contains at least one item anchored to the question (an excerpt or document title carrying a specific question term) and the drafted core answer is lexically supported by it.

Verification

The independent LLM stage that judges whether each claim of a drafted answer is ENTAILED by, CONTRADICTED by, or UNKNOWN relative to the retrieved evidence.

Fail-closed boundary

The enforced rule that an answer with no supported claims is replaced by the explicit insufficient-information response and its citations cleared.

Structured knowledge

Versioned, authoritative rule tables (salary, travel allowances, procurement thresholds) registered by the application and evaluated deterministically — plus a generic store for validated rules, thresholds and tables.

Rule-first (RULE_ENGINE) path

The routing strategy in which structured knowledge resolves the question exactly; the LLM only explains the computed decision. Retrieval is skipped.

Qdrant

Vector database storing chunk embeddings (collection `mda_chunks`, 768-d, cosine) and serving similarity search.

Neo4j

Graph database storing the enriched document graph and serving keyword + traversal queries.

PostgreSQL

Relational database storing documents, chunks, users, audit events — and serving the keyword retrieval path.

Ollama

Local model server hosting the generation, verifier and embedding models on the deployment host.

References and Source Notes

This white paper is grounded in direct inspection of the Verwaltungsassistent implementation (August 2026; refreshed September 2026). Key implementation areas referenced in this document:

- **Reasoning core** (platform-ai-runtime) — single execution path (AiService), routing (DecisionRouter), retrieval planning (RetrievalPlanner), domain gate (DomainGate), evidence selection (EvidencePackageBuilder), prompt construction (DefaultPromptBuilder, template compact-v9), claim verification (DefaultClaimVerifier), grounding and the fail-closed boundary (DefaultGroundingService), semantic intent parsing (LlmSemanticIntentParser / RegexSemanticIntentParser).
- **Search services** (platform-search-runtime, platform-search-qdrant, platform-neo4j) — hybrid fusion (DefaultHybridRetrievalService), keyword search (JpaKeywordSearchProvider), vector search (QdrantVectorSearchProvider), graph adapter (Neo4jGraphSearchAdapter), indexing orchestration (DefaultIndexingOrchestrationService).
- **Reference deployment** (verwaltungsassistent-web) — municipal domains and structured tables (MunicipalDomains, MunicipalKnowledgeInitializer), structured-knowledge extraction (StructuredKnowledgeExtractionService, KnowledgeItemValidator), domain terminology (domain-knowledge.yml), ingestion processor, email intake (MailboxIngestionService, CommunicationClassifier, IncomingCommunicationAnalysisService), security configuration.
- **Demo corpus** — BRKG, AV zu § 55 LHO Berlin, Bauordnung Berlin, 15 official Berlin Service-Portal procedure documents (registration, identity documents, driver's licence, certifications, eID; retrieved 2026-08-15), plus the versioned knowledge base (GWB Part 4, TV-L 2025, Bauvorlagenverordnung Berlin).
- **Infrastructure** — PostgreSQL 16 (pgvector image), Qdrant, Neo4j 5, local Ollama model serving; Docker Compose profiles and a reproducible on-premise VM bundle (docs/deployer/).

Named components (Qdrant, Neo4j, PostgreSQL, Ollama, PDFBox, Apache Tika) are the trademarks of their respective owners and are referenced descriptively. All measurements are labelled development measurements unless stated otherwise.